

Bagging Regressor

Contents

1. Overview	2
2. The core idea	2
3. Seeing it on a curve	3
4. BaggingRegressor on the Boston housing data	4
4.1 Three single regressors	4
4.2 A bagging regressor with default settings	4
4.3 Tuning with GridSearchCV	4
4.4 The out-of-bag score	5
5. Summary	6
6. Key terms	6

Where this fits: Pipeline map steps 8 (Model), 9 (Evaluate), 10 (Tune). See the [Course map](#).



- **Builds on:** Simple imputation (mean, median, mode, constant) ([Note 37](#)); ML pipelines ([Note 38](#)); Bias-variance trade-off ([Note 62](#)); Overfitting ([Note 91](#)); Hyperparameter tuning ([Note 98](#)); Decision trees ([Note 100](#)).
- **Leads to:** Random forest ([Note 108](#)).
- **Compare with:** Boosting ([Note 101](#)); Voting ensembles ([Note 104](#)); Cross-validation ([Note 104](#)); Random forest ([Note 108](#)); Optuna ([Note 134](#)); Bayesian optimisation ([Note 134](#)).

1. Overview

Key point: A bagging regressor works exactly like a bagging classifier, except that aggregation takes the mean of the base models' numbers instead of a majority vote.

The bagging regressor applies bagging (the [bagging Note](#)) to regression. Everything carries over from the [bagging classifier Note](#): the four sampling types and every hyperparameter. This Note covers:

- the one difference, in the aggregation step;
- a demo with curves: one tree against a bagged ensemble;
- `BaggingRegressor` on the Boston housing data, tuned with `GridSearchCV`.

The Notebook (`notebook.ipynb`) runs every experiment. The Dash app `app.py` lets us change the base model and the bagging settings and redraws both curves.

2. The core idea

Key point: Bootstrapping is unchanged; aggregation averages the predictions.

In the **bootstrapping** step, each base model is trained on its own random sample of the data, as before. So pasting, random subspaces and random patches are all available too.

In the **aggregation** step, each base model returns a number, since this is regression. The bagging regressor returns their **mean**. That is the only change.

scikit-learn's `BaggingRegressor` has exactly the same hyperparameters as `BaggingClassifier` (the [bagging classifier Note](#), section 6): `estimator`, `n_estimators`, `max_samples`, `max_features`, `bootstrap`, `bootstrap_features` and `oob_score`.

3. Seeing it on a curve

Key point: One fully grown tree passes through every training point (test R^2 0.92); 50 bagged trees give a smooth curve that follows the pattern (0.95).

The demo data has one input column. Its output is two Gaussian bumps (a small one at 0 and a larger one at 2) plus a little noise, with 150 training points and 100 test points. With a single input there are no columns to sample, so the app only offers row settings: the base model (decision tree, SVR or KNN), `n_estimators`, `max_samples` and `bootstrap`.

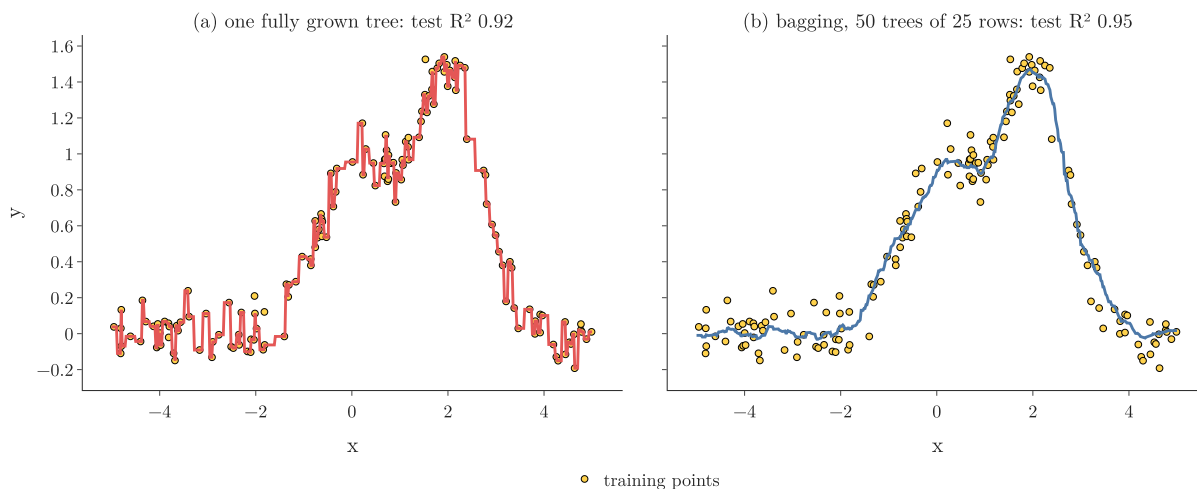


Figure 1: (a) One fully grown regression tree; (b) a bagging regressor of 50 trees, each trained on 25 rows drawn with replacement

Figure 1 compares a single decision tree with bagging (50 trees, 25 rows each, with replacement):

- **(a) The single tree** tries to pass through every training point, outliers included: low bias, high variance, so it overfits. Test R^2 : **0.92**.
- **(b) The bagging regressor** gives a smooth curve. It does not touch every point; it follows the underlying pattern. Test R^2 : **0.95**.

This is the same effect as in classification: bagging keeps the low bias and removes much of the variance.

Changing the settings makes little difference on this easy data: 100 trees score 0.952, and pasting (75 rows, `bootstrap=False`) scores 0.947. Bagging is not always an improvement, though: bagged KNN (0.832) and bagged SVR (0.871) both do worse than a single KNN (0.954) or SVR (0.894) here.

4. BaggingRegressor on the Boston housing data

Key point: Out of the box, bagged trees (test R^2 0.83) beat linear regression (0.66), a single tree (0.38) and KNN (0.55).

4.1 Three single regressors

Key point: Linear regression 0.66, decision tree 0.38, KNN 0.55 on one test split.

The Boston housing data (the [regression trees Note](#), section 7.1) has 506 districts, 13 input columns and the median house price. We split it into 404 training rows and 102 test rows, and train three single regressors:

Model	Linear regression	Decision tree	KNN
Test R^2	0.659	0.380	0.548

4.2 A bagging regressor with default settings

Key point: Ten bagged trees, nothing tuned: training R^2 0.98, test R^2 0.83.

Python: A bagging regressor with every setting at its default.

```
from sklearn.ensemble import BaggingRegressor
```

```
bag = BaggingRegressor(random_state=1)
bag.fit(X_train, y_train)
bag.score(X_train, y_train)    # 0.980
bag.score(X_test, y_test)     # 0.828
```

The defaults: `estimator=None` (a decision tree), `n_estimators=10`, every training row per model (`max_samples=None`), drawn with replacement, and every column. `score` returns R^2 for a regressor.

Without tuning anything, the bagging regressor beats all three single models. The gap between training (0.98) and test (0.83) shows some overfitting remains.

4.3 Tuning with GridSearchCV

Key point: A grid search over 144 combinations picks 50 bagged trees on full bootstrap samples; cross-validation R^2 0.87, test R^2 0.81.

Instead of trying pasting, random subspaces and random patches by hand, we let `GridSearchCV` try them all (the [KNN Note](#), section 4.2):

Python: Searching over the base model and the sampling settings.

```
params = {
    "estimator": [None, LinearRegression(),
```

```

        KNeighborsRegressor()],
    "n_estimators": [20, 50, 100],
    "max_samples": [0.5, 1.0],
    "max_features": [0.5, 1.0],
    "bootstrap": [True, False],
    "bootstrap_features": [True, False]}
grid = GridSearchCV(BaggingRegressor(random_state=1),
                    param_grid=params, cv=3,
                    n_jobs=-1, verbose=1)
grid.fit(X_train, y_train)
grid.best_params_

```

The base model itself is a hyperparameter here: `None` stands for the default decision tree. With `bootstrap` and `bootstrap_features` both tried as `True` and `False`, the search covers bagging, pasting, random subspaces and random patches.

That is $3 \times 3 \times 2 \times 2 \times 2 = 144$ combinations, each with 3-fold cross-validation: 432 fits, 13 seconds with `n_jobs=-1`. The best settings:

- base model: **decision tree**;
- `bootstrap=True`: **bagging**, not pasting;
- `bootstrap_features=False`, `max_features=1.0`: **no column sampling**;
- `max_samples=1.0`: every model gets as many rows as the training set;
- `n_estimators=50`.

Their cross-validation R^2 is **0.871**, and the test R^2 is **0.806**.

Extra: The 0.871 is an average over the three validation folds of the training data; the 0.806 is measured on the 102 test rows. The two differ because they are measured on different rows, and 102 rows is a small, noisy test. With 100 different random splits, the same settings score anywhere from 0.63 to 0.94 on the test rows (mean 0.857, close to the 0.871 from cross-validation), so one 102-row test score can land far from the average. Expecting the test score to match `best_score_` exactly is a common mistake. Here the tuned model scores slightly lower on this test set than the default one (0.828); the search picks settings that do best on average, not on this particular test set. Over the same 100 splits, the tuned settings beat the default on 79 (mean 0.857 against 0.841).

4.4 The out-of-bag score

Key point: With `oob_score=True`, the regressor reports R^2 on the out-of-bag rows: 0.870 here.

`oob_score` works for regression too (the [bagging classifier Note](#), section 4). For a regressor, `oob_score_` is the R^2 on the rows each model never saw. With 50 trees it is **0.870**, close to the cross-validation score.

5. Summary

	Bagging classifier	Bagging regressor
Base models	classifiers	regressors
Aggregation	majority vote	mean
scikit-learn	<code>BaggingClassifier</code>	<code>BaggingRegressor</code>
score and <code>oob_score_</code>	accuracy	R^2
Hyperparameters	<code>estimator</code> , <code>n_estimators</code> , <code>max_samples</code> , <code>max_features</code> , <code>bootstrap</code> , <code>bootstrap_features</code> , <code>oob_score</code>	the same

- Only aggregation changes for regression: the mean replaces the vote.
- On the bumps data, bagging smooths one overfitting tree into a curve that follows the pattern: test R^2 0.92 to 0.95.
- On Boston, default bagged trees score 0.83 against 0.66 for the best single model; a grid search picks plain bagging of 50 trees (cross-validation 0.87).

6. Key terms

Term	Meaning
Bagging regressor	A bagging ensemble of regressors that predicts the mean of their predictions
<code>BaggingRegressor</code>	scikit-learn class for bagging, pasting, random subspaces and random patches in regression